

A background image of four students in a library. A young woman with long dark hair is on the left, smiling. Next to her is a young man with dark hair, also smiling. To his right is a young woman with glasses and dark hair, looking towards the right. On the far right is a young man with dark hair, seen from the back/side, looking at a laptop. They are all sitting at a table with books and a laptop. The background is filled with bookshelves.

Java 25

Module Import Declarations

Module Import Declarations

- A new import form: *import module M*;
 - This brings in on-demand all public top-level types from packages exported by module *M*.
 - Your code does not have to be part of a module (no *module-info.java* needed).
- This reduces import “noise” - fewer import statements at the top of the file.
- Module Import Declarations and Compact Source Files were co-developed at the same time.
 - Both are beginner-friendly and great for quick prototyping.
 - In fact, compact source files implicitly `import module java.base;`



Module Import Declarations

- Importing multiple modules can lead to simple-name clashes e.g. *java.awt.List* versus *java.util.List*.
 - This is fixed by explicitly importing the specific type you desire (a single-type import take precedence over module imports), or use the fully qualified name).
- Module Import Declarations are a compile-time name-resolution feature and thus, your code does not bloat due to loads of extra imports.
 - In other words, at compile-time, you are widening the search path for type names (and not copying whole libraries into your class file).

